CancellationToken

и конкурентность в C# / .NET

В чем отличие конкурентности от параллелизма?

Что такое токен отмены

CancellationToken – структура, которая представляет уведомление об отмене некоторого действия.

Является частью Task Parallel Library (TPL) и особенно полезна в асинхронном коде где необходимо «чисто*» завершить долгоживущую операцию.

Отмена in .NET – **кооперативна**: одна часть кода запрашивает отмену, а другая часть реагирует на этот запрос.

Реагирующий код <u>может</u> сразу остановить выполнение операции, продолжить выполнение пока не достигнет какой-то валидной точки для безопасного прерывания или может полностью проигнорировать запрос.

В одних ситуациях, токены отмены предоставляются библиотеками:

- ASP.NET предоставляет CancellationToken представляющий неожиданное прерывание клиентского соединения
- Polly предоставляет делегат с CancellationToken представляющий обобщенную ситуацию (например, вызов политики таймаута).

В других, есть необходимость в создании самими разработчиками:

- Можно использовать конструктор класса CancellationToken или значение CancellationToken.None для создания токена, который всегда в состоянии переданного или не переданного сигнала, и в другом состоянии не будет
- В большинстве случаев токен создается, чтобы быть отмененным позже. Для этого используется тип CancellationTokenSource:

```
1  using var cts = new CancellationTokenSource();
2
3  await LongAwaitedOperation(cts.Token);
```

Каждый объект CancellationTokenSource контролирует свое множество токенов отмены – объектов типа CancellationToken

Каждый объект CancellationToken – всего лишь небольшая структура, которая связана с CancellationTokenSource

Токен отмены CancellationToken может отвечать только на запросы отмены

Объект CancellationTokenSource необходим для запроса отмены

Таким образом, код, который запрашивает отмену некоторой операции создает объект CancellationTokenSource и хранит ссылку на него.

Отвечающий событию отмены код, получает объект CancellationToken и использует его для ответа на событие.

Высвобождение ресурсов

Класс CancellationTokenSource имплементирует IDisposable интерфейс.

Всегда вызывайте метод Dispose или используйте using statement / блок для автоматического вызова метода и высвобождения используемых ресурсов.

При высвобождении CancellationTokenSource очищаются связанный таймер (если есть) и коллбэки.

Очистка CancellationTokenSource происходит когда выполняется отмена или вызывается Dispose.

⚠️ Вы можете вызывать **отмену** или **высвобождение** источника токена, но убедитесь, что любой метод был вызван чтобы избежать утечки ресурсов. В некоторых ситуациях отмена более удобна.

Сценарии

Большинство случаев

В метод-получатель добавляется параметр CancellationToken и объект пробрасывается вниз по логике:

```
1  async Task DoSomethingAsync(int data, CancellationToken cancellationToken)
2  {
3      var intermediateValue = await DoFirstStepAsync(data, cancellationToken);
4      await DoSecondStepAsync(intermediateValue, cancellationToken);
5  }
```

CancellationToken может сигнализировать о любой отмене:

- Пользователь нажал кнопку
- Клиент отсоединился от сервера
- Остановка приложения
- Таймаут

Для принимающего кода не должно быть важным почему это происходит. Важен сам факт отмены.

Контракт отмены: объявление метода

По соглашению, параметр CancellationToken – обычно последний параметр в объявлении метода если в нем нет параметра IProgress<T>.

Стандартная практика – передавать перезагруженный или параметр по умолчанию, чтобы вызывающему коду не пришлось предоставлять CancellationToken если он отмена не подразумевается; значение default – это CancellationToken.None, т.е., токен отмены никогда не будет отменен.

```
1   Task DoSomethingAsync(int data) => DoSomethingAsync(data, CancellationToken.None);
2
3   async Task DoSomethingAsync(int data, CancellationToken cancellationToken)
4   {
5       ...
6   }
7
8   // or:
9
10  async Task DoSomethingAsync(int data, CancellationToken cancellationToken = default)
11  {
12      ...
13  }
```

В своем коде не рекомендуется передавать отдельными параметрами CancellationToken и значение таймаута. Такой код можно увидеть в BCL для более эффективных вызовов.

Объявляя параметр CancellationToken, метод неявно говорит о том, что он может (но не обязан) обработать событие отмены.

В некоторых случаях (например, имплементации интерфейса), аргумент CancellationToken может быть проигнорирован.

Отмена операции

```
// Уведомить об отмене операции синхронно
cts.Cancel();

// Уведомить об отмене операции асинхронно
await cts.CancelAsync();

// Уведомить об отмене операции синхронно с задержкой
cts.CancelAfter(5000);
```

Можно определить токен, который отменяется автоматически через какое-то время:

```
using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(10));
```

Каждый токен отмены может быть отменен только один раз, т.е. всегда находится в состоянии отмены.

Исключение из большинства случаев

🛑 Не стоит передавать CancellationToken в Task.Run

Семантика вызова не очевидна:

- когда выполняется отмена токена, не происходит отмена выполнения делегата
- CancellationToken нужен для отмены планирования задачи в пуле потоков
- как только делегат начинается выполняться (почти сразу), CancellationToken игнорируется

```
async Task DoSomethingAsync(CancellationToken cancellationToken)
{
   var test = await Task.Run(() =>
   {
       // логика делегата
       // IDE может подсвечивать, что cancellationToken не используется,
       // поэтому внутри делегата стоит этот токен принимать во внимание
   });
   ...
}
```

Таймауты

Один из частых сценариев использования токена отмены – реализация таймаута. Решение сводится к таймеру, который запрашивает отмену по истечению времени.

Сценарий насколько частый, что у CancellationTokenSource есть встроенное поведение. Используйте либо конструктор класса для передачи времени таймера или вызывайте .CancelAfter() у существующего объекта CancellationTokenSource.

```
async Task DoSomethingWithTimeoutAsync()

{
   // Создать токен, который вызовет отмену через 5 минут.
   using CancellationTokenSource cts = new(TimeSpan.FromMinutes(5));

   // Передать токен в асинхронный метод.
   await DoSomethingAsync(cts.Token);

   // Перед выходом из метода cts будет высвобожден.
   // С этого момента не стоит использовать его токены.
}
```

Ручная отмена

Для более общих случаев у объекта CancellationTokenSource есть метод Cancel для ручной отмены токена. Например, при нажатии кнопки “Отменить” в приложении.

```
private CancellationTokenSource? _cts;

async void StartButton_Click(..)
{
   using var cts = _cts = new();

   try
   {
       await DoSomethingAsync(_cts.Token);
   }
   catch (Exception ex)
   {
       // обработать и отобразить ошибку
   }
}

async void CancelButton_Click(..)
{
   _cts!.Cancel();
}
```

Возврат из метода в случае отмены

🛑 В Интернете много примеров кода с `return` при запросе отмены действия. Такой подход нарушает контракт отмены! Когда отвечающий на запрос код делает ранний `return`, вызывающий код не знает был ли запрос учтен или проигнорирован.

Поллинг

Стандартный паттерн – периодически вызывать метод .ThrowIfCancellationRequested().

```
void DoSomething(CancellationToken cancellationToken)
{
   while (!done)
   {
       cancellationToken.ThrowIfCancellationRequested();
       Thread.Sleep(200); // выполняем блокирующую операцию
   }
}
```

Приведенный код проверяет токен отмены **до** выполнения действия, что является в целом хорошей практикой, т.к. возможна ситуация когда переданный токен уже отменен на момент начала выполнения операции.

ThrowIfCancellationRequested выполняет уведомление об отмене за разработчика; наш код должен только дать возможность всплыть исключению выше по стеку вызова.

Как часто пулить? Универсального ответа нет.

В идеале нужно стараться пулить как можно реже в течение секунды. Эмпирическим путем определяем подходящий тайминг 🤷🏼‍♂️

Еще пример:

```
void DoSomethingForever(CancellationToken cancellationToken)
{
   while (true)
   {
       cancellationToken.ThrowIfCancellationRequested();
       Thread.Sleep(200);
   }
}
```

Когда пулить?

1. Когда есть блокирующий код, например, CPU-bound операции.
2. Когда есть “дорогие операции”, продолжение выполнения которых не требуется в случае отмены

P.S. У Parallel и PLINQ есть поддержка токенов отмены.

```
async Task LongAwaitedOperation(CancellationToken token)
{
   try
   {
       for (int i = 0; i < 100; i++)
       {
           token.ThrowIfCancellationRequested();
           await Task.Delay(1000, token);
       }
   }
   catch (OperationCanceledException)
   {
       Console.WriteLine("Выполнение операции было отменено");
   }
}
```

Как не надо делать

```
void DoSomethingForever(CancellationToken cancellationToken)
{
   Environment.FailFast("Плохой код, не делайте так!");

   while (!cancellationToken.IsCancellationRequested)
   {
       Thread.Sleep(200); // блокирующий вызов
   }
}
```

При выполнении метода периодически выполняется проверка отмены токена, но когда отмена запрошена, метод просто делает ранний возврат → контракт отмены не соблюдается! Вызывающий метод код не знает завершился ли в итоге метод или был отменен.

Заметка про асинхронный код

```
async Task DoComplexWorkAsync(CancellationToken cancellationToken)
{
   cancellationToken.ThrowIfCancellationRequested(); // <-
   await DoStep1Async(cancellationToken);

   cancellationToken.ThrowIfCancellationRequested(); // <-
   await DoStep2Async(cancellationToken);
}
```

Если нет уверенности, что вызванные методы учитывают токен отмены, можно делать явную проверку на каждом из этапов.

Регистрация коллбэков

Регистрация коллбэк позволяет получить коллбэк сразу при запросе отмены токена. Коллбэк может выполнить операции (обычно вызывая сторонний код) для отмены асинхронного кода метода.

```
async Task DoSomethingAsync(CancellationToken cancellationToken)

{
   using var registration = cancellationToken.Register(() => StopSomething());
   StartSomething();
   await SomethingCompletedTask;
}
```

Метод Register возвращает регистрацию токена отмены, что по сути является просто объектом типа IDisposable.

🛑 Т.к. обработка коллбэков не всегда выполняется предсказуемым образом, не рекомендуется выбрать исключения в коллбэке.

Возможные проблемы

Что случится если токен отмены будет отменен во время регистрации коллбэка (в том же потоке)? Эту ситуацию можно обработать по следующему правилу: если коллбэк добавлен для токена отмены, который уже отменен, он должен быть вызван немедленно и синхронно.

Всегда высвобождайте объект по завершению асинхронной операции:

```
async Task DoSomethingAsync(CancellationToken cancellationToken)
{
   using var registration = cancellationToken.Register(() => StopSomething());
   StartSomething();
   await SomethingCompletedTask;
} // registration высвобождается здесь если операция завершена без отмены
```

```
async Task LongAwaitedOperation(CancellationToken token)
{
   HttpClient client = new();

   token.Register(() => {
       client.CancelPendingRequests();
       Console.WriteLine("Операция отменена");
   });

   var result = await client.GetStringAsync("https://planfact.io");
}
```

Блокирующий вызов коллбэков

🛑 Любые зарегистрированные коллбэки **сразу же синхронно** вызываются при выполнении метода Cancel прежде чем произойдет выход из него.

Это может стать причиной дедлоков или любого другого поведения если код написан с расчетом на то, что коллбэки вызываются после завершения метода Cancel. Поэтому коллбэки не должны выполнять никаких блокирующих операций!

В .NET 8.0 добавлен метод CancellationTokenSource.CancelAsync, который вызывает коллбэки в пуле потоков. Технически он сразу же и синхронно переводит cancellation token source в состояние отмены и отправляет в очередь задачи на выполнение коллбэков в пул потоков. Задачи завершаются когда завершается выполнение коллбэков. Также вероятна ситуация когда `CancelAsync` сделает return до завершения коллбэков если уже был вызван CancelAsync.

Связанные токены отмены

```
async Task DoWorkWithMultipleTokensAsync(CancellationToken externalToken)
{
   using var timeoutCts = new CancellationTokenSource(TimeSpan.FromSeconds(5));
   using var linkedCts = CancellationTokenSource.CreateLinkedTokenSource(externalToken, timeoutCts.Token);

   try
   {
       await DoWorkAsync(linkedCts.Token);
   }
   catch (OperationCanceledException)
   {
       Console.WriteLine("Операция была отменена или превышено время ожидания.");
   }
}
```

```
async Task ExecuteWithTimeoutAsync(CancellationToken cancellationToken)
{
   ResiliencePipeline pipeline = new ResiliencePipelineBuilder()
       .AddTimeout(TimeSpan.FromSeconds(10))
       .Build();

   await pipeline.ExecuteAsync(async token =>
   {
       // ...
   }, cancellationToken);
}
```

```
async Task MainAsync()
{
   using var cts = new CancellationTokenSource();
   cts.CancelAfter(TimeSpan.FromSeconds(2000));

   try
   {
       await DoSomethingAsync(cts.Token);
   }
// не делайте так!
// catch (OperationCanceledException ex) when (ex.CancellationToken == cts.Token)
   catch (OperationCanceledException) when (cts.IsCancellationRequested)
   {
       Console.WriteLine("Timeout!");
   }
   catch (Exception ex)
   {
       Console.WriteLine(ex);
   }
}
```

Как отвечать на **запрос** отмены

```
async Task TryDoSomethingAsync()
{
   using CancellationTokenSource cts = new();

   // ...
   // какой-то код, который может запросить отмену операции

   try
   {
       await DoThingAsync(cts.Token); // прокидываем токен отмены в операцию
   }
   catch (Exception ex) when (ex is not OperationCanceledException)
   {
       // подразумеваем, что асинхронный метод соблюдает контракт
       // обрабатываем исключение – пишем в лог и т.д.
   }
}
```

```
async Task DoSomethingAsync()
{
   using CancellationTokenSource cts = new();

   // ...
   // какой-то код, который может запросить отмену операции


   try
   {
       await DoThingAsync(cts.Token); // прокидываем токен отмены в операцию
   }
   catch (OperationCanceledException)
   {
       // Обрабатываем событие отмены операции
   }
   catch (Exception ex)
   {
       // Обрабатываем исключение, которые возникло во время выполнения операции
   }
}
```

TaskCanceledException и OperationCanceledException.CancellationToken

🛑 Не ловите исключение TaskCanceledException. Ловите родительский тип – OperationCanceledException.

🛑 Если нужно понять что запрос отмена случилась благодаря нашему токену, а не каким-то другим образом, вместо

```
catch (OperationCanceledException ex) when (ex.CancellationToken == cts.Token)
```

стоит писать

```
catch (OperationCanceledException ex) when (cts.IsCancellationRequested)
```

```
1 usage  Станислав Струченков +3
private async Task HandleQueue(CancellationToken cancellationToken)
{
    while (!cancellationToken.IsCancellationRequested)
    {
        await _onTaskEnqueue.WaitAsync(cancellationToken);
        while (_taskQueue.TryDequeue(out var generationTaskId))
        {
            if (_taskDictionary.TryGetValue(generationTaskId, out var generationTask))
            {
                await TryGenerateEntries(generationTask, cancellationToken).ConfigureAwait(false);
            }
            else
            {
                // Этого не может быть, поэтому с грохотом падаем
                throw new CommonBusinessException( title: $"EntryGenerationTaskId {generationTaskId} not found");
            }
        }
    }
}
```

Для практики: ревью кода ПФ

```kotlin
fun main() = runBlocking {
   // Create an HTTP client
   val client = HttpClient(CIO)

   // Create a parent Job to control cancellation
   val cancellationJob = Job()
   val scope = CoroutineScope(Dispatchers.IO + cancellationJob)

   val requestJob = scope.launch {
       try {
           println("Starting HTTP request...")
           val response: HttpResponse = client.get("https://jsonplaceholder.typicode.com/posts") {
               // Optional: Configure request timeout or headers here
           }
           println("Response received: ${response.status}")
           val responseBody = response.bodyAsText()
           println("Response body: ${responseBody.substring(0, 100)}...") // Print only first 100 chars
       } catch (e: CancellationException) {
           println("Request was cancelled: ${e.message}")
       } catch (e: Exception) {
           println("Error during HTTP request: ${e.message}")
       } finally {
           println("Closing HTTP client...")
           client.close()
       }
   }

   // Simulate some work and then cancel the job
   delay(100) // Simulate some delay
   println("Cancelling HTTP request...")
   cancellationJob.cancel() // Cancel the job

   requestJob.join() // Wait for the request to finish
   println("Program finished")
}
```

```go
func main() {
    // Create a context with a timeout
    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
    defer cancel() // Ensure resources are cleaned up

    // Create the HTTP request with context
    req, err := http.NewRequestWithContext(ctx, http.MethodGet, "https://jsonplaceholder.typicode.com/posts", nil)
    if err != nil {
        fmt.Printf("Failed to create request: %v\n", err)
        return
    }

    client := &http.Client{}
    fmt.Println("Starting HTTP request...")

    // Execute the request
    resp, err := client.Do(req)
    if err != nil {
        // Check if the error is due to context cancellation or timeout
        if ctx.Err() == context.DeadlineExceeded {
            fmt.Println("Request timed out")
        } else if ctx.Err() == context.Canceled {
            fmt.Println("Request was canceled")
        } else {
            fmt.Printf("Request failed: %v\n", err)
        }
        return
    }
    defer resp.Body.Close()

    // Read and display the response
    body, err := ioutil.ReadAll(resp.Body)
    if err != nil {
        fmt.Printf("Failed to read response: %v\n", err)
        return
    }

    fmt.Printf("Response received: %s\n", string(body[:100])) // Print only the first 100 characters
}
```